

Bachelorarbeit

Exposé
Predicting Open-Source Dependency Inactivity Risk
for Secure Software Supply Chains

Zur Veranstaltung... / zur Erlangung des akademischen Grades

'Bachelor of Science' B.Sc. / 'Master of Science' M.Sc.

im Studiengang Bachelor-Studiengang Informatik

vorgelegt dem Fachbereich

Informatik und Wirtschaftsinformatik / Wirtschaftswissenschaften
der

Provadis School of International Management and Technology

von

Eric Krause

Matrikelnummer: D720

BIN-T23-F-4

eric.krause@telekom.de

Erstgutachter: Prof. Dr. Lamya Abdullah

Zweitgutachter: [Second Reviewer]

Ende der Bearbeitungsfrist: [Date]

Contents

1	Topic and relevance	3
2	Research question.....	4
3	Practical context	4
4	Methodology	5
5	Theoretical foundations and key literature	6
6	Time Plan	6
7	Draft outline.....	7
8	References (IEEE).....	9

i. Glossary

Term	Definition / Explanation
AUROC	Area Under the Receiver Operating Characteristic Curve; threshold-independent classification metric.
Brier score	Proper scoring rule for probabilistic predictions; mean squared error between predicted probabilities and outcomes.
CHAOSS	Community Health Analytics Open Source Software; community providing standardized OSS health metrics models.
deps.dev	Public service/dataset providing dependency metadata and package-to-repository mappings (Open Source Insights).
GH Archive	Public dataset of GitHub event streams used for scalable, historical activity analysis.
ML	Machine learning; statistical methods that learn patterns from data for prediction.
ORT	OSS Review Toolkit; tool suite (incl. Analyzer) to extract dependencies from project manifests/lockfiles.
OSI	Open Source Initiative; steward of the Open Source Definition.
OSS	Open-source software; software published under an OSI-compliant license.
PR-AUC	Area Under the Precision-Recall Curve; metric suited for imbalanced classification.
Scorecard	OpenSSF Scorecard; automated checks for repository security practices used as potential indicators.
SLSA	Supply-chain Levels for Software Artifacts; framework/specification for software supply-chain integrity.
SPDX	Software Package Data Exchange; standard identifiers for licenses and package metadata.
SSDF	NIST Secure Software Development Framework; guidance for secure software development practices.

1 Topic and relevance

Modern software systems rely heavily on open-source software (OSS) components, making dependency management an important part of secure software development. Inactive dependencies can increase operational and security-related risk for downstream users, for example through delayed updates, reduced responsiveness to issues, and higher internal maintenance effort [1].

Against this background, this thesis investigates whether the inactivity risk of OSS dependencies can be estimated within a prediction horizon of 12 months. Inactivity is treated not as a definitive indicator of project death, but as an operational proxy for elevated maintenance risk. The 12-month horizon is intended to reduce noise from short temporary pauses while remaining relevant for dependency triage and monitoring [3].

The proposed approach uses publicly observable repository signals and standardized health metrics, and may be complemented by security-practice indicators such as OpenSSF Scorecard checks [4], [6]. Because prior research shows that context-free health indicators do not automatically reflect sustainability across heterogeneous OSS projects, the results will be interpreted conservatively [5].

2 Research question

To what extent can a machine-learning-based inactivity-risk scoring tool, using only publicly observable repository health indicators and security-practice signals, predict 12-month OSS dependency inactivity and support risk-based dependency triage in secure dependency management?

3 Practical context

The practical context of this thesis is secure dependency management in real-world software projects that rely on numerous OSS dependencies, including transitive dependencies, where third-party component risk is a recurring concern in secure development practice [1]. In such settings, teams must decide which dependencies require additional monitoring, review, or replacement planning—decisions that benefit from actionable, measurable signals rather than ad-hoc judgement.

As an applied component, the thesis demonstrates a lightweight workflow that derives a project’s dependency set and maps packages to source repositories using Open Source Insights / deps.dev resources [7], [8], including the corresponding dataset interfaces where applicable [11], [19]. The resulting repository list is scored by the trained model to produce a ranked inactivity-risk view that can support triage and monitoring decisions.

4 Methodology

This thesis follows a Design Science Research-oriented approach with empirical evaluation. The central artifact is an inactivity-risk scoring workflow for OSS dependencies that combines repository-level data collection, feature construction, model training, and risk-oriented output for dependency triage [20], [21]. The methodological design is grounded in prior research on OSS inactivity and survival-style operationalization, literature on OSS health metrics and their limitations, and software supply-chain security guidance together with security-practice indicators [1]–[6], [13]–[16].

The work begins with a structured literature review supported by snowballing. The aim is to identify relevant approaches to OSS inactivity operationalization and predictors, standardized OSS health metrics, and supply-chain security guidance and security-practice indicators [1]–[6], [13], [14], [16]. On this basis, the primary outcome variable will be defined as repository inactivity within the interval $[t, t+12 \text{ months}] / [t, t+12 \text{ months}]$, operationalized mainly through the absence of commits and releases or tags. This label is used as a proxy for elevated maintenance risk rather than as a universal definition of project abandonment. For that reason, issue and pull-request activity are treated as predictor variables instead of label components. Responsiveness-related indicators will be interpreted cautiously because prior work has shown that such measurements can be distorted, for example by bot-generated responses [3], [15].

The empirical dataset will be constructed from publicly accessible OSS repositories with an OSI-compliant license and a detectable SPDX-based or equivalent license reference, using GitHub metadata and deps.dev sources where appropriate [17]–[19]. To reduce noise, forks, mirrors, and repositories with very low signal will be excluded. Repository-level and issue or pull-request data will be collected from public GitHub sources. Where suitable, scalable public datasets and interfaces such as GH Archive and Open Source Insights or deps.dev will be used to improve reproducibility and coverage, while GitHub REST or GraphQL APIs may supplement missing repository attributes [10]–[12], [19]. Reproducibility will be supported by documenting extraction steps, queries, timestamps, snapshot definitions, and the exact sources used.

Feature engineering will focus on activity, maintenance, and selected CHAOSS-aligned health indicators, and may be extended by security-practice features derived from OpenSSF Scorecard checks [4], [6]. For model development, simple baselines such as a recency heuristic and logistic regression will be compared with tabular machine-learning models such as random forest or gradient boosting. A time-aware split will be used to reduce temporal leakage and to align the evaluation with the snapshot-based prediction setting [3]. Model quality will then be assessed using the discrimination and calibration criteria defined in the research

question, and results will be interpreted conservatively in light of the known limitations of cross-project health indicators [5], [22], [23].

As a practical demonstrator, the thesis will additionally extract a project’s direct and transitive dependencies using OSS Review Toolkit and map them to source repositories through Open Source Insights or deps.dev resources [7]–[9], [11], [19]. The trained model will then be applied to this dependency set in order to generate a ranked inactivity-risk view. This demonstrator is intended to show how the proposed artifact could support dependency monitoring and risk-based triage in software supply-chain management.

Evaluation objectives: Predictive discrimination will be assessed using the area under the receiver operating characteristic curve (AUROC) and the area under the precision-recall curve (PR-AUC). Because the expected class distribution may be imbalanced, particular attention will be given to PR-AUC [22]. The approach will be compared against a simple recency-based baseline, and calibration quality will additionally be assessed using Brier score and calibration curves to determine whether the predicted probabilities are usable for triage [23]. An AUROC around 0.80 is treated as a practical orientation value rather than a strict success threshold.

5 Theoretical foundations and key literature

The research is anchored in four strands of literature and guidance: Software supply chain security guidance and integrity frameworks: NIST SSDF [1] and SLSA [2] (motivation and security context). OSS health metric models: CHAOSS Starter Project Health Metrics Model [4] (operational, measurable indicators). Inactivity/survival modeling and metric limitations: survival-style operationalization and covariates [3], and critiques of context-free cross-project health indicators [5]. Security-practice signals: OpenSSF Scorecard and empirical evaluation of its checks [6].

Dependency graph data sources for applied evaluation: Open Source Insights / `deps.dev` documentation and dataset access [7], [8]. Empirical and practitioner work on project health metrics and their use in decision-making: overview of CHAOSS and standardized health measurement transparency [13], and applied guidance using CHAOSS Starter metrics for assessing OSS health [14], [16]. Measurement caveat for responsiveness: bot-generated first responses can distort “Time to First Response” and should be separated from human responses where possible [15].

6 Time Plan

[illegible]

Literature review	X	X	X	X								
Definitions & dataset design			X	X								
ORT + deps.dev mapping				X	X							
Data collection & preprocessing					X	X						
Feature engineering						x	X	X				
Modeling (baseline + ML)								X	X			
Evaluation									X			
Demonstrator + case study									X	X	X	
Writing (continuous)	X	X	X	X	X	X	X	X	X	X	X	X
Final editing & submission											X	X

7 Draft outline

Proposed table of contents for the final thesis (for discussion):

Front matter

Title page

Abstract

Table of contents

List of figures / tables

List of abbreviations

1 Introduction

1.1 Motivation and problem statement

1.2 Research question and objectives

1.3 Scope and assumptions

1.4 Contributions

1.5 Structure of the thesis

2 Background

- 2.1 Software supply chain security
- 2.2 OSS sustainability and project health
- 2.3 Metric pitfalls and limitations
- 2.4 Machine learning basics for tabular risk scoring

3 Related work

- 3.1 OSS inactivity / abandonment operationalizations
- 3.2 Repository health metrics and dashboards
- 3.3 Security practice indicators

4 Method

- 4.1 Definitions and inactivity label
- 4.2 Dataset construction and time-based snapshot design
- 4.3 Data acquisition
- 4.4 Feature engineering
- 4.5 Models and baselines
- 4.6 Evaluation plan

5 Implementation

- 5.1 Data pipeline and storage schema
- 5.2 Reproducibility

6 Results and discussion

- 6.1 Predictive performance
- 6.2 Calibration and threshold selection
- 6.3 Feature importance and ablation results
- 6.4 Error analysis and case studies

7 Practical demonstrator

- 7.1 Example dependency set and mapping
- 7.2 Risk ranking output and interpretation guidance

8 Threats to validity

9 Conclusion and outlook

References

Appendix

Queries

Additional plots

Dataset documentation

AI statement / Declaration of honour

8 References (IEEE)

- [1] M. Souppaya, K. Scarfone, and D. Dodson, “Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities,” NIST Special Publication 800-218, Feb. 2022, doi: 10.6028/NIST.SP.800-218.
- [2] SLSA Community, “SLSA Specification v1.2,” Nov. 2025. [Online]. Available: <https://slsa.dev/spec/v1.2/>.
- [3] D. Robinson, K. Enns, N. Koulecar, and M. Sihag, “Two Approaches to Survival Analysis of Open Source Python Projects,” arXiv:2203.08320, 2022.
- [4] CHAOSS Community, “Metrics Model: Starter Project Health,” accessed Feb. 2026. [Online]. Available: <https://chaoss.community/kb/metrics-model-starter-project-health/>.
- [5] Y. Yehudi, C. Goble, and C. Jay, “Individual context-free online community health indicators fail to identify open source software sustainability,” arXiv:2309.12120, 2023.
- [6] N. Zahan, P. Kanakiya, B. Hambleton, S. Shohanuzzaman, and L. Williams, “OpenSSF Scorecard: On the Path Toward Ecosystem-wide Automated Security Metrics,” arXiv:2208.03412, 2022.
- [7] Google, “Using the Open Source Insights dataset to analyze open source dependencies,” Google Cloud Blog, Nov. 18, 2022. [Online]. Available: <https://cloud.google.com/blog/topics/developers-practitioners/using-the-open-source-insights-dataset>.
- [8] Google, “Open Source Insights / deps.dev documentation,” accessed Feb. 2026. [Online]. Available: <https://docs.deps.dev/>.
- [9] OSS Review Toolkit (ORT), “Analyzer,” documentation, accessed Feb. 2026. [Online]. Available: <https://oss-review-toolkit.github.io/ort/docs/tools/analyzer>.
- [10] I. Grigorik, “GH Archive: public GitHub timeline dataset,” accessed Feb. 2026. [Online]. Available: <https://www.gharchive.org/> and <https://github.com/igrigorik/gharchive.org>.

-
- [11] Open Source Insights (deps.dev), “BigQuery dataset documentation,” accessed Feb. 2026. [Online]. Available: <https://docs.deps.dev/bigquery/v1/>.
- [12] GitHub, “REST and GraphQL API rate limits,” documentation, accessed Feb. 2026. [Online]. Available: <https://docs.github.com/en/rest/using-the-rest-api/rate-limits-for-the-rest-api> and <https://docs.github.com/en/graphql/overview/rate-limits-and-query-limits-for-the-graphql-api>.
- [13] S. P. Goggins, M. Germonprez, and K. Lombard, “Making Open Source Project Health Transparent,” *Computer*, vol. 54, no. 8, Aug. 2021, doi: 10.1109/MC.2021.3084015.
- [14] J. Colt, “An introduction to using metrics to assess the health and sustainability of library open source software projects,” *The Code4Lib Journal*, no. 57, Aug. 2023. [Online]. Available: <https://journal.code4lib.org/articles/17514>.
- [15] K. A. Hasan, M. Macedo, Y. Tian, B. Adams, and S. H. H. Ding, “Understanding the Time to First Response in GitHub Pull Requests,” in *Proc. IEEE/ACM 20th Int. Conf. on Mining Software Repositories (MSR)*, 2023. [Online]. Available: <https://arxiv.org/abs/2304.08426>.
- [16] D. Foster, “Building Healthy and Sustainable Open Source Projects,” *Computer*, 2025, doi: 10.1109/MC.2025.3530556.
- [17] Open Source Initiative (OSI), “The Open Source Definition,” accessed Feb. 2026. [Online]. Available: <https://opensource.org/osd>.
- [18] GitHub, “REST API endpoints for licenses,” documentation, accessed Feb. 2026. [Online]. Available: <https://docs.github.com/en/rest/licenses/licenses>.
- [19] deps.dev, “deps.dev API (Open Source Insights),” documentation, accessed Feb. 2026. [Online]. Available: <https://docs.deps.dev/api/v3/>.
- [20] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design Science in Information Systems Research,” *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.
- [21] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, “A Design Science Research Methodology for Information Systems Research,” *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007.
- [22] T. Saito and M. Rehmsmeier, “The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets,” *PLOS ONE*, vol. 10, no. 3, e0118432, 2015.
- [23] A. Niculescu-Mizil and R. Caruana, “Predicting Good Probabilities With Supervised Learning,” in *Proc. 22nd Int. Conf. on Machine Learning (ICML)*, 2005, pp. 625–632.
